



FLASHYIELDS

Flash Loan and Rebate Contracts

Security Assessment Report

Version: 3.3

June, 2026

Contents

Introduction	2
Disclaimer	2
Document Structure	2
Overview	2
Security Assessment Summary	3
Scope	3
Approach	3
Coverage Limitations	4
Findings Summary	4
Detailed Findings	5
Summary of Findings	6
Public Deposits Restore Revoked Vault Flash Loan Allowances	7
Rebate Minimum Lock Duration Can Be Shortened By <code>veToken</code> Rounding	9
Partial <code>claimReward()</code> Token Lists Can Strand Rewards	11
Vault Credits Nominal Deposits For Non-Standard Tokens	13
Best-Effort Filtering Can Still Revert	15
A Vulnerability Severity Classification	16

Introduction

Sigma Prime was commercially engaged to perform a time-boxed security review of the FlashYields components in scope. The review focused solely on the security aspects of the Solidity implementation of the contracts, though general recommendations and informational comments are also provided.

Disclaimer

Sigma Prime makes all effort but holds no responsibility for the findings of this security review. Sigma Prime does not provide any guarantees relating to the function of the components in scope. Sigma Prime makes no judgements on, or provides any security review, regarding the underlying business model or the individuals involved in the project.

Document Structure

The first section provides an overview of the functionality of the FlashYields components contained within the scope of the security review. A summary followed by a detailed review of the discovered vulnerabilities is then given which assigns each vulnerability a severity rating (see [Vulnerability Severity Classification](#)), an *open/closed/resolved* status and a recommendation. Additionally, findings which do not have direct security implications (but are potentially of interest) are marked as *informational*.

The appendix provides additional documentation, including the severity matrix used to classify vulnerabilities within the FlashYields components in scope.

Overview

FlashYields is a flashloan protocol.

The scope of this review focused on Solidity contracts, helper libraries, and interfaces for the FlashYields flash-loan, vault, and rebate workflows, particularly:

- `ApprovalFlashLoan.sol` implements the flash-loan coordinator. It transfers ERC20 assets from approved lenders to borrower receivers, supports direct ERC20 allowances and Permit2-based transfers, provides best-effort lender filtering, and verifies that principal and fees are repaid.
- `FeeVerifier.sol` calculates and verifies flash-loan fees. It supports default and per-asset fee rates, alternate fee-token configuration, price-based fee conversion, and fee-recipient balance checks.
- `ERC2oVault.sol` aggregates user deposits into a vault address that can act as a lender to the flash-loan contract. It tracks depositor balances per ERC20 token and manages the vault's flash-loan allowances.
- `Rebate.sol` is a Merkle-based reward distributor for multi-token rebate claims. Depending on the configured category, claimants can receive rewards directly or stake rewards into the voting-escrow token.

Security Assessment Summary

Scope

The review was conducted on the files hosted on the [flash-yields/flashyields-contracts](#) repository.

The scope of this time-boxed review was strictly limited to the following files, assessed at commit [027f9ce](#):

- `interfaces/IACManager.sol`
- `interfaces/IAllowanceTransfer.sol`
- `interfaces/IApprovalFlashLoan.sol`
- `interfaces/IApprovalFlashLoanReceiver.sol`
- `interfaces/IEACAggregatorProxy.sol`
- `interfaces/IFeeVerifier.sol`
- `interfaces/IRebate.sol`
- `interfaces/IVotingEscrow.sol`
- `interfaces/IWETH.sol`
- `ApprovalFlashLoan.sol`
- `ERC20Vault.sol`
- `FeeVerifier.sol`
- `Rebate.sol`
- `libraries/InputValidator.sol`

The fixes to the FlashYields components were assessed at commit [027f9ce](#).

After the review, the contracts were deployed at:

1. Rebate: [0x4D71B255963DaC711506020B5139402b168B3418](#)
2. FeeVerifier: [0x7F8C574610b1EA0C333E5d2681e2916403284FFf](#)
3. ApprovalFlashLoan: [0x51a6262916c1fDD5dD25E3bA11cbF76427A126A3](#)

During the assessment, an issue was identified in a component outside the agreed deployment scope and not intended for production release. To avoid ambiguity in the public-facing assessment regarding the security posture of the deployed system, the finding has been moved to a separate informational report provided directly to the client.

Note: third party libraries, dependencies, deployment scripts, mocks, examples, and generated artifacts were excluded from the scope of this assessment.

Approach

The security assessment covered components written in Solidity.

The manual review focused on identifying issues associated with the business logic implementation of the contracts. This includes their internal interactions, intended functionality and correct implementation with respect to the underlying functionality of the Ethereum Virtual Machine (for example, verifying correct storage/memory layout).

Additionally, the manual review process focused on identifying vulnerabilities related to known Solidity anti-patterns and attack vectors, such as re-entrancy, front-running, integer overflow/underflow and correct visibility specifiers.

To support the Solidity components of the review, the testing team may use the following automated testing tools:

- Aderyn: <https://github.com/Cyfrin/aderyn>
- Slither: <https://github.com/trailofbits/slither>
- Mythril: <https://github.com/ConsenSys/mythril>

Output for these automated tools is available upon request.

Coverage Limitations

Due to the time-boxed nature of this review, all documented vulnerabilities reflect best effort within the allotted, limited engagement time. As such, Sigma Prime recommends to further investigate areas of the code, and any related functionality, where majority of critical and high risk vulnerabilities were identified.

Findings Summary

The testing team identified a total of 5 issues during this assessment. Categorised by their severity:

- Medium: 1 issue.
- Low: 3 issues.
- Informational: 1 issue.

Detailed Findings

This section provides a detailed description of the vulnerabilities identified within the FlashYields components in scope. Each vulnerability has a severity classification which is determined from the likelihood and impact of each finding by the matrix given in the Appendix: [Vulnerability Severity Classification](#).

A number of additional properties of the components, including optimisations, are also described in this section and are labelled as “informational”.

Each vulnerability is also assigned a **status**:

- **Open:** the finding has not been addressed by the project team.
- **Resolved:** the finding was acknowledged by the project team and updates to the affected components have been made to mitigate the related risk.
- **Closed:** the project team has reviewed and acknowledged the finding, but the recommended remediation has not been implemented. The project team has typically provided a documented rationale supporting this decision, including the approach taken and any associated risk considerations.

Summary of Findings

ID	Description	Severity	Status
FLY-01	Public Deposits Restore Revoked Vault Flash Loan Allowances	Medium	Resolved
FLY-02	Rebate Minimum Lock Duration Can Be Shortened By veToken Rounding	Low	Resolved
FLY-03	Partial <code>claimReward()</code> Token Lists Can Strand Rewards	Low	Closed
FLY-04	Vault Credits Nominal Deposits For Non-Standard Tokens	Low	Resolved
FLY-05	Best-Effort Filtering Can Still Revert	Informational	Closed

FLY-01	Public Deposits Restore Revoked Vault Flash Loan Allowances		
Assets	contracts/ERC20Vault.sol		
Status	Resolved: See Resolution		
Rating	Severity: Medium	Impact: Medium	Likelihood: Medium

Description

Any account able to deposit a nonzero amount of the token can restore a revoked vault-to-flash-loan allowance by making a small deposit of the revoked token, bypassing the admin's intended control over whether vault-held assets are available for flash loans.

ERC20Vault aggregates lenders into a single lender address, and exposes admin functions to explicitly approve or revoke the vault's token allowance to the flash-loan contract.

However, `deposit()` is publicly callable and also automatically grants the flash-loan contract an unlimited allowance whenever the current allowance is zero:

contracts/ERC20Vault.sol::deposit()

```
function deposit(address[] calldata tokens, uint256[] calldata amounts) external nonReentrant {
    if (tokens.length != amounts.length) revert ArrayLengthMismatch();

    for (uint256 i = 0; i < tokens.length; i++) {
        if (amounts[i] == 0) revert ZeroAmount();

        balances[tokens[i]][msg.sender] += amounts[i];
        IERC20(tokens[i]).safeTransferFrom(msg.sender, address(this), amounts[i]);

        if (IERC20(tokens[i]).allowance(address(this), flashLoan) == 0) {
            // @audit Any public depositor can recreate max allowance after admin revocation.
            IERC20(tokens[i]).forceApprove(flashLoan, type(uint256).max);
        }

        emit Deposited(msg.sender, tokens[i], amounts[i]);
    }
}
```

There is no separate revocation flag checked by `deposit()`. Therefore, after an admin revokes a vault allowance, any account that can transfer a nonzero amount of that same token to the vault can call `deposit()` and trigger the approval path again.

As a result, an attacker can execute the following sequence:

1. The admin calls `revokeTokensFromFlashloan()` for a token that should no longer be borrowable from the vault.
2. The attacker deposits a minimal amount of the same token through `deposit()`.
3. The deposit path observes that the allowance is zero and restores `type(uint256).max` allowance to the flash-loan contract.
4. The vault's full token balance can again be used as flash-loan liquidity, unless the token is also separately black-listed in `ApprovalFlashLoan`.

The impact is medium because this breaks the intended admin-controlled lending boundary for vault assets and can re-enable flash-loan access to assets that were explicitly revoked. The likelihood is medium because the attack requires a token to be revoked first, but after that condition is met, exploitation only requires a public deposit of the same token.

Recommendations

Remove allowance management from the public `deposit()` path.

Vault-to-flash-loan approvals should only be created or restored through the admin-only `approveTokensForFlashloan()` function, and `revokeTokensFromFlashloan()` should remain final until an admin explicitly re-approves the token.

Resolution

The development team removed flash loan allowance management from `ERC2oVault.deposit()` and moved vault-to-flash-loan approvals to the admin-controlled `setTokenWhitelist()` and `approveTokensForFlashloan()` paths. The fix can be found in commit [027f9ce](#).

FLY-02	Rebate Minimum Lock Duration Can Be Shortened By veToken Rounding		
Assets	contracts/Rebate.sol		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Medium

Description

`Rebate.claimReward()` validates the user-provided `_initUnlockTime` before forwarding it to the configured `veToken`, which rounds lock end times down to whole-week boundaries. A user can therefore satisfy the rebate contract's minimum duration check while receiving a shorter actual lock than the rebate category intended to enforce.

`Rebate` allows the bookkeeper to configure a per-category `minUnlockDuration` via `updateMerkleRootInfo()`. When a user without an existing lock claims a staked reward, `claimReward()` checks the raw `_initUnlockTime` against this minimum and then passes the same raw timestamp to `veToken.create_lock_for()`:

`contracts/Rebate.sol::claimReward()`

```

} else if (l.amount == 0) {
    if (_initUnlockTime <= block.timestamp) {
        revert InvalidUnlockTime();
    }
    if (_initUnlockTime - block.timestamp < uint256(merkleRootInfo[_category].minUnlockDuration)) {
        // @audit - validates raw timestamp, not the rounded value veToken will actually store
        revert InvalidUnlockTime();
    }
    veToken.create_lock_for(msg.sender, totalClaimAmount, _initUnlockTime); // @audit - veToken rounds _initUnlockTime down to a
    ↪ week boundary
    emit RewardStaked(msg.sender, totalClaimAmount, _initUnlockTime);
} else {
    revert CannotStakeForExpiredLock();
}

```

`veToken` rounds `_unlock_time` down to the previous whole-week boundary before storing the lock. This means the rebate contract validates a raw timestamp that differs from the actual stored lock end. For example, if `minUnlockDuration` is 7 days and a user claims shortly before a whole-week boundary, the user can pass `_initUnlockTime = block.timestamp + 7 days`. That timestamp falls shortly before the following whole-week boundary, so `veToken` rounds it down to the intervening whole-week boundary, which may be only seconds after the claim. The rebate contract accepts the 7-day minimum as met, while the actual lock is much shorter.

For longer minimum durations, the bypass is bounded by the week-rounding window. A one-year minimum can still be shortened by up to almost 7 days.

This issue has a low impact as it is a lock-policy bypass rather than direct fund loss, but it weakens any reward, rebate, or governance-fairness assumption that relies on `minUnlockDuration` being the actual stored commitment period. This issue has a medium likelihood as it requires no privileged access and only needs a claim timed near a whole-week boundary, though it does require an active Merkle category with staking enabled.

Recommendations

Round `_initUnlockTime` down using the same week-boundary logic as `veToken` before comparing against `minUnlockDuration`, and require the rounded value to remain greater than `block.timestamp`, so the check reflects the stored lock value.

Resolution

The development team updated `Rebate.claimReward()` to round `_initUnlockTime` down to the `veToken` week boundary before validating the minimum unlock duration and creating the lock. The fix can be found in commit [027f9ce](#).

FLY-03	Partial <code>claimReward()</code> Token Lists Can Strand Rewards		
Assets	contracts/Rebate.sol		
Status	Closed: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

Users can strand part of a valid Merkle reward claim under the current root by omitting a token from the user-supplied `_tokens` array. The omitted rewards remain in `Rebate`, but require an admin-published replacement root to become claimable again.

`Rebate.claimReward()` requires users to claim all tokens and sub-wallet rewards for a root in a single call. The contract comment documents this requirement, but the function does not enforce that `_tokens` covers every token present in `_entries`.

On a successful call, the function marks the root as claimed for the user while aggregating claim amounts only for tokens that appear in `_tokens`. Entries whose token is absent from `_tokens` are still included in the Merkle leaf, but are not accumulated into any payable `TokenClaim` entry:

contracts/Rebate.sol::claimReward()

```

if (rewardClaimed[msg.sender][_category][merkleRoot[_category]]) {
    revert RewardAlreadyClaimed();
}
// @audit Root marked claimed before any tokens are distributed.
rewardClaimed[msg.sender][_category][merkleRoot[_category]] = true;

TokenClaim[] memory tokenClaims = new TokenClaim[](_tokens.length);
for (uint256 i = 0; i < _tokens.length; i++) {
    tokenClaims[i].token = _tokens[i];
}

for (uint256 i = 0; i < _entries.length; i++) {
    packed = bytes.concat(packed, abi.encodePacked(
        _entries[i].user,
        _entries[i].token,
        _entries[i].claimAmount
    ));
    for (uint256 j = 0; j < _tokens.length; j++) {
        if (tokenClaims[j].token == _entries[i].token) {
            tokenClaims[j].totalClaimAmount += _entries[i].claimAmount;
            break;
        }
    }
    // @audit Entries whose token is not in _tokens are silently skipped.
}

for (uint256 i = 0; i < _tokens.length; i++) {
    if (tokenClaims[i].totalClaimAmount == 0) {
        continue;
    }
    IERC20(_tokens[i]).safeTransfer(msg.sender, tokenClaims[i].totalClaimAmount);
}

```

For example, if a valid Merkle leaf entitles a claimant to 100 USDC and 50 WETH, the claimant can accidentally call `claimReward()` with `_tokens = [USDC]`. The proof still verifies against both entries, the claimant receives only the USDC, and the claim is marked as complete for that root. A later attempt to claim the omitted WETH reverts with `RewardAlreadyClaimed()`.

This issue has a low impact as no attacker can directly steal the omitted rewards, the tokens remain in the rebate contract, and the admin can make the missing amount claimable again by publishing a new Merkle root. This issue has a low likelihood because `_tokens` is expected to be supplied by the project frontend or claim tooling, but a direct caller or integration error can still omit part of a multi-token allocation.

Recommendations

Validate that `_tokens` covers every token present in `_entries` before marking the root as claimed. Prefer deriving the unique token list directly from `_entries` instead of accepting a separate user-supplied `_tokens` filter, so every verified entry is either paid or the transaction reverts.

Resolution

The development team has closed the issue with the following comment:

For FLY-03, we acknowledge the finding.

FLY-04	Vault Credits Nominal Deposits For Non-Standard Tokens		
Assets	contracts/ERC20Vault.sol		
Status	Resolved: See Resolution		
Rating	Severity: Low	Impact: Low	Likelihood: Low

Description

The vault credits users for the nominal deposit amount instead of the amount actually received, so unsupported non-standard ERC20 tokens can make internal accounting exceed the vault's real token balance.

ERC20Vault accepts arbitrary token addresses in `deposit()` and tracks user balances in an internal `balances[token][user]` mapping. The contract does not maintain a token allowlist, and the function comments describe deposits as supporting multiple ERC20 token addresses.

The deposit path increments the user's internal balance before calling `safeTransferFrom()`:

contracts/ERC20Vault.sol::deposit()

```
function deposit(address[] calldata tokens, uint256[] calldata amounts) external nonReentrant {
    if (tokens.length != amounts.length) revert ArrayLengthMismatch();

    for (uint256 i = 0; i < tokens.length; i++) {
        if (amounts[i] == 0) revert ZeroAmount();

        // @audit User is credited with the requested amount, not the received amount.
        balances[tokens[i]][msg.sender] += amounts[i];
        IERC20(tokens[i]).safeTransferFrom(msg.sender, address(this), amounts[i]);

        if (IERC20(tokens[i]).allowance(address(this), flashLoan) == 0) {
            IERC20(tokens[i]).forceApprove(flashLoan, type(uint256).max);
        }

        emit Deposited(msg.sender, tokens[i], amounts[i]);
    }
}
```

This accounting is unsafe for fee-on-transfer, rebasing, or otherwise non-standard ERC20 tokens. For example, if a token takes a transfer fee, a user who deposits 100 tokens may only transfer 90 into the vault while still receiving an internal credit of 100. The `withdraw()` path trusts the inflated balance, so the vault becomes internally insolvent for that token once the accounting gap is exposed.

This issue has a low impact as this is an unsupported-token compatibility issue. This issue has a low likelihood as it depends on users depositing a non-standard token that is already incompatible with the intended flash-loan flow.

Recommendations

Implement a token allowlist that controls which tokens can be deposited and receive flash-loan approval from the vault. As defence in depth, compare the vault's token balance before and after each transfer and credit only the amount actually received.

Resolution

The development team added an admin-controlled token whitelist for `ERC20Vault` deposits and flash-loan approvals, and updated `ERC20Vault.deposit()` to credit users with the actual amount received by the vault. The fixes can be found in commit [027f9ce](#).

FLY-05	Best-Effort Filtering Can Still Revert	
Assets	contracts/ApprovalFlashLoan.sol	
Status	Closed: See Resolution	
Rating	Informational	

Description

Best-effort flash loan modes can still revert because filtering is performed per entry. The filter only checks allowance for each requested row, and does not account for insufficient lender balances or cumulative exposure from duplicate lender entries:

```
contracts/ApprovalFlashLoan.sol::_filterValidateAndSnapshot()
for (uint256 i = 0; i < lenders.length; i++) {
    if (amounts[i] == 0) revert AmountMustBePositive();
    if (blacklistedTokens[assets[i]]) {
        continue;
    }

    IERC20 asset = IERC20(assets[i]);
    if (asset.allowance(lenders[i], address(this)) < amounts[i]) {
        // @audit - only checks allowance, not balance or cumulative usage
        continue;
    }

    validLenders[validCount] = lenders[i];
    validAmounts[validCount] = amounts[i];
    validCount++;
}
```

Lenders that pass the filter can still make the later transfer helpers revert if their live balance, or duplicate-row cumulative exposure, is insufficient.

Recommendations

Consider documenting that best-effort modes can still revert if repeated lender entries exceed the lender's available balance or allowance.

Resolution

The development team updated the `ApprovalFlashLoan` best-effort filters to skip lender entries when the lender's live token balance is lower than the requested amount. The development team acknowledged that duplicate entries for the same lender and token can still exceed the lender's cumulative balance or allowance, and left this edge case unaddressed with the following comment:

The borrower is responsible for filtering input parameters off-chain, which also saves gas by reducing checks.

The fix can be found in commit [027f9ce](#).

Appendix A Vulnerability Severity Classification

This security review classifies vulnerabilities based on their potential impact and likelihood of occurrence. The total severity of a vulnerability is derived from these two metrics based on the following matrix.

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Table 1: Severity Matrix - How the severity of a vulnerability is given based on the *impact* and the *likelihood* of a vulnerability.

σ'